NAME: **SONI CHAUDHARY**

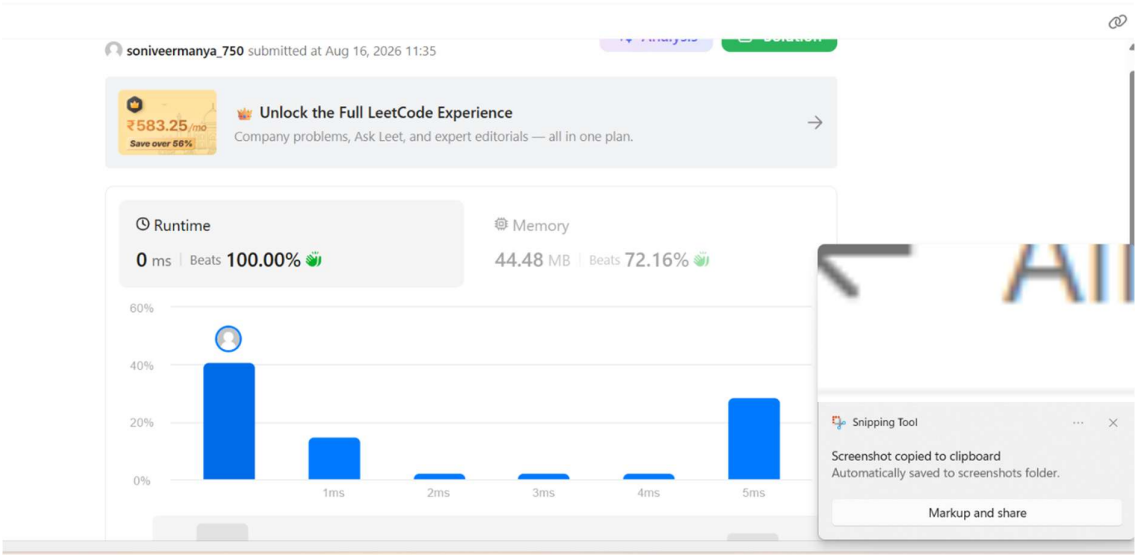
UNIVERSITY ADMISSION NO:

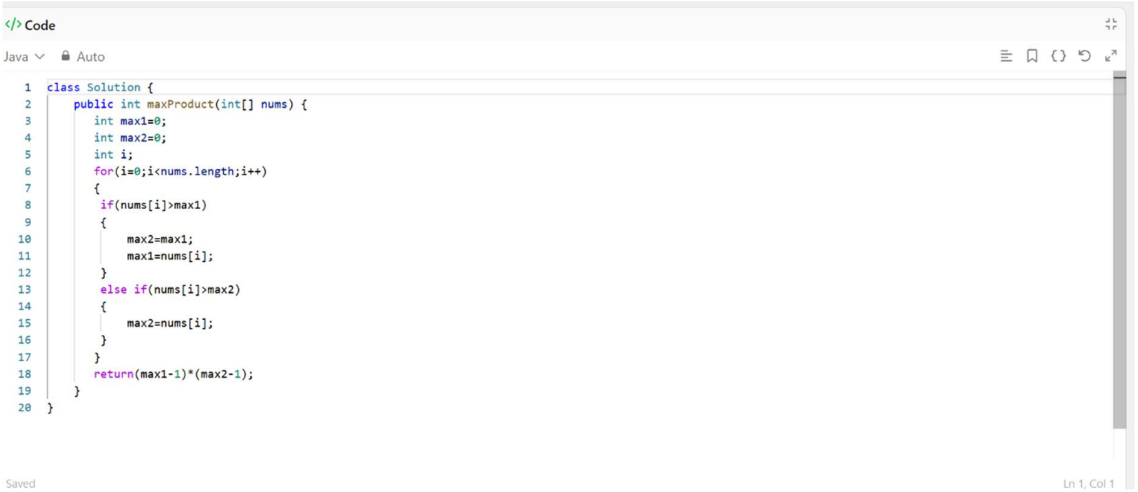
12515001612

CLASS ROLL NO: **62**

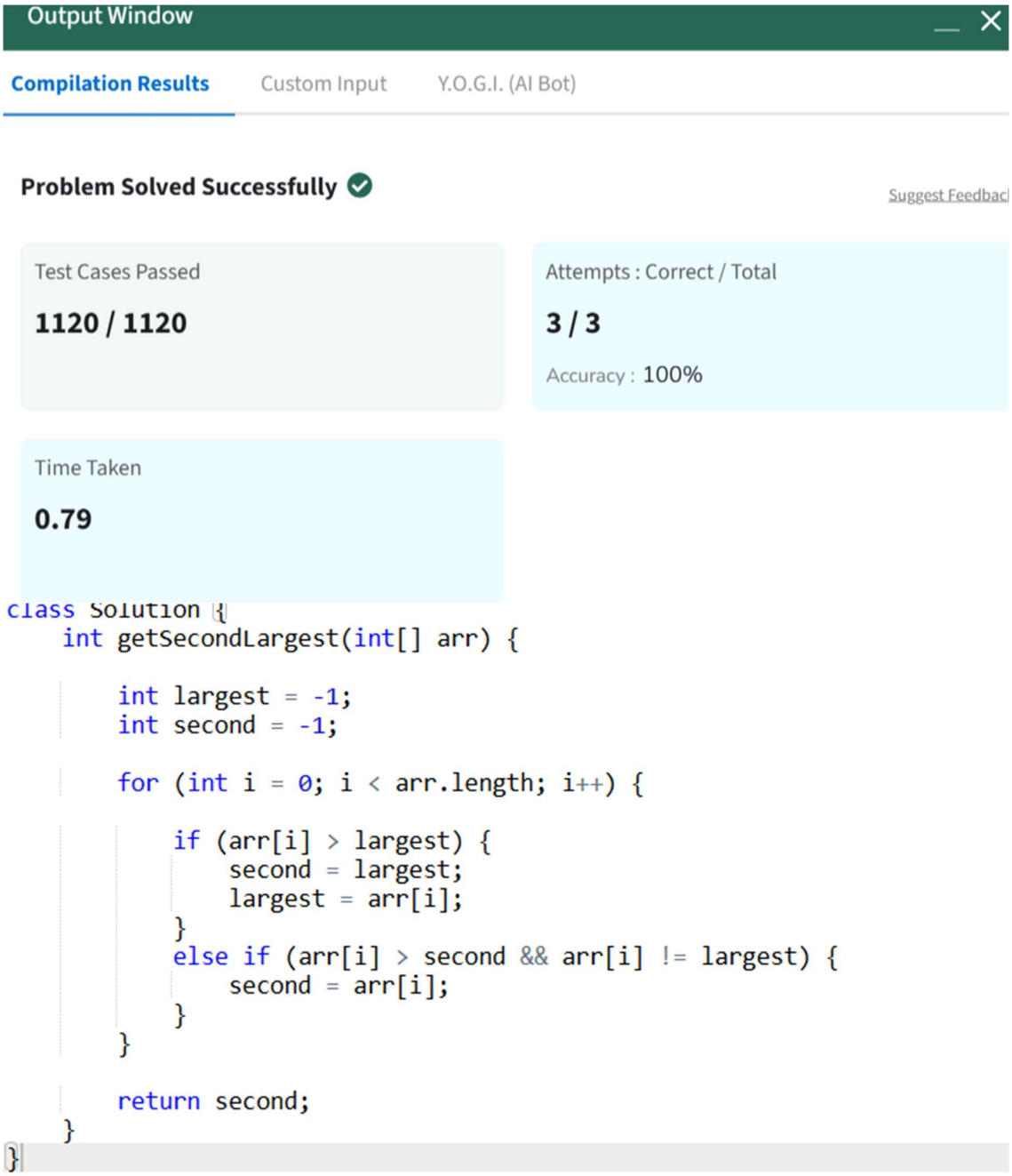
SECTION: **FB**

1. Maximum Product of Two Elements in an Array



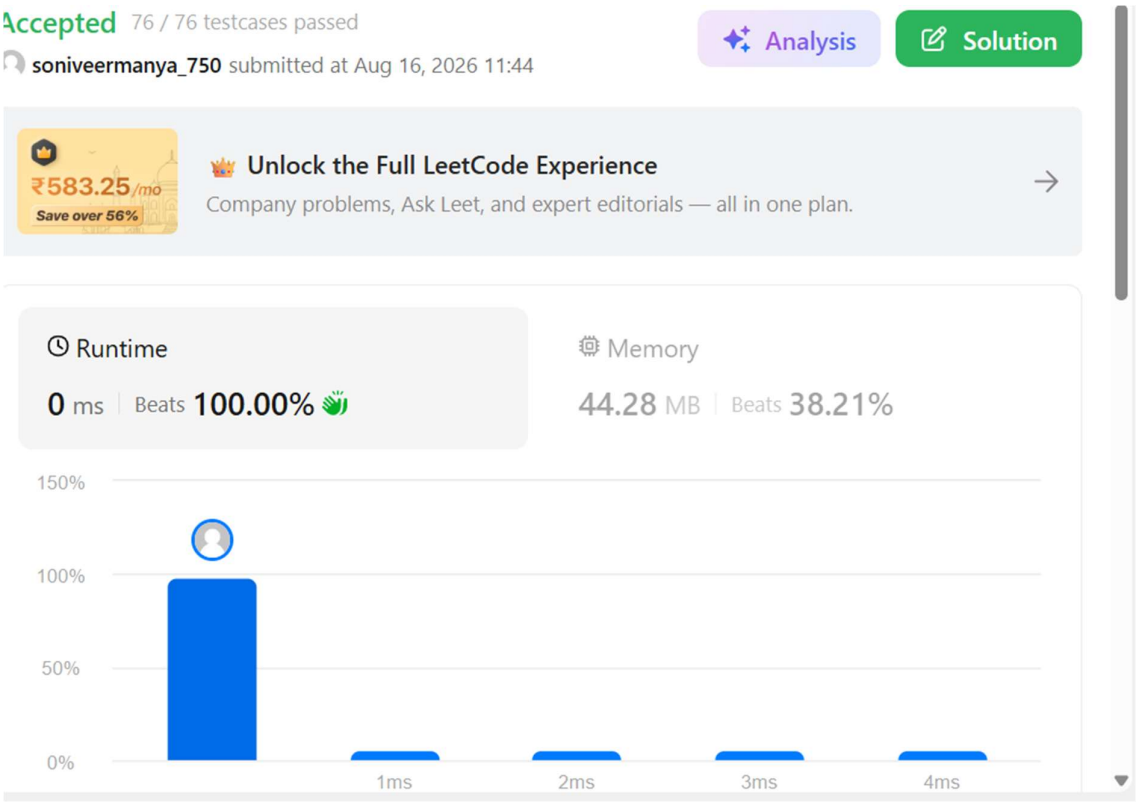


2. Second-Largest



3. Find Peak Element

```java
class Solution {
    public int findPeakElement(int[] nums) {
        int i,n=0;
        int max=nums[0];
        for(i=1;i<nums.length;i++)
        {
            if(nums[i]>max)
            {
                max=nums[i];
                n=i;
            }
        }
    return n;
    }
}
```



4. Search in Rotated Sorted Array

```java
class Solution {
    public int search(int[] nums, int target) {

        int left = 0;
        int right = nums.length - 1;

        while (left <= right) {

            int mid = left + (right - left) / 2;

            if (nums[mid] == target) {
                return mid;
            }

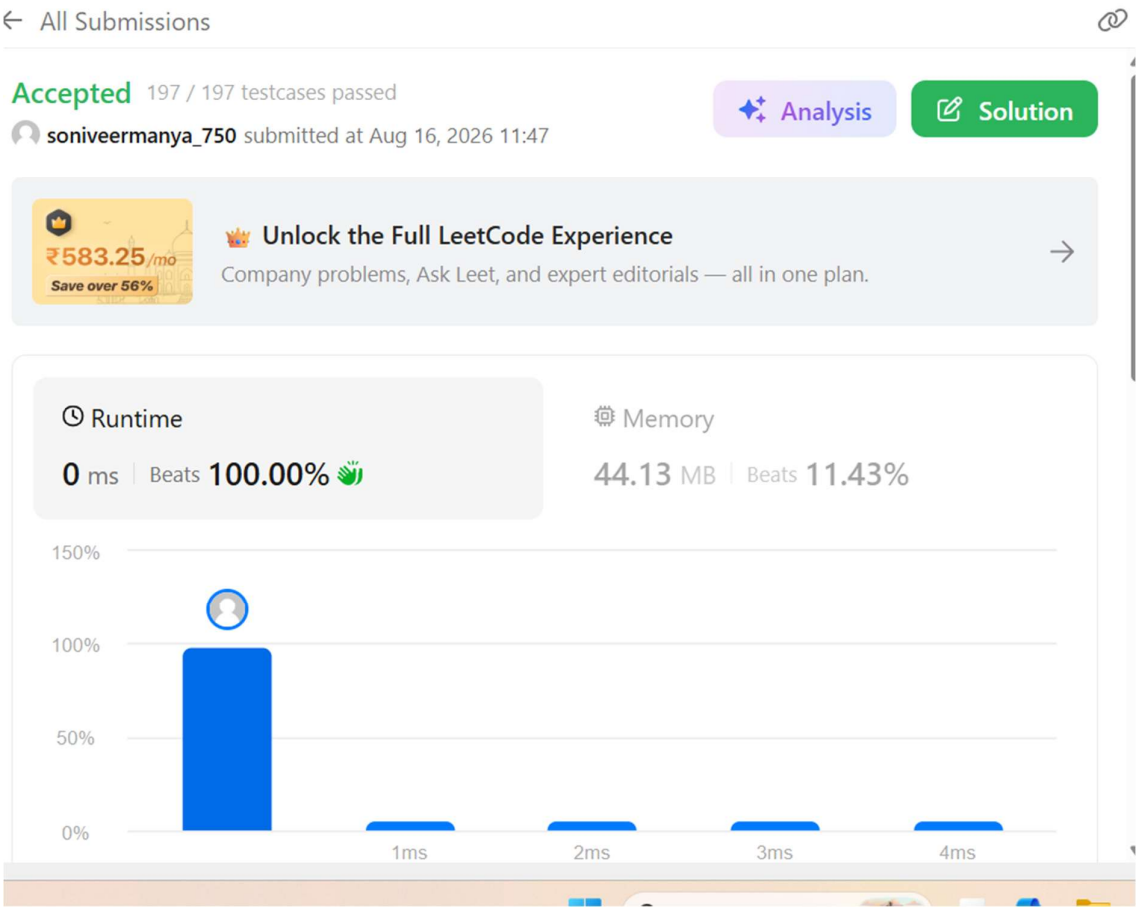
            // Left half is sorted
            if (nums[left] <= nums[mid]) {

                if (nums[left] <= target && target < nums[mid]) {
                    right = mid - 1;
                }
                else {
                    left = mid + 1;
                }
            }

            // Right half is sorted
            else {

                if (nums[mid] < target && target <= nums[right]) {
                    left = mid + 1;
                }
                else {
                    right = mid - 1;
                }
            }
        }

        return -1;
    }
}
```



5. Fruits Into Baskets II

```java
class Solution {
    public int numOfUnplacedFruits(int[] fruits, int[] baskets) {

        int unplaced = 0;

        for (int i = 0; i < fruits.length; i++) {

            boolean placed = false;

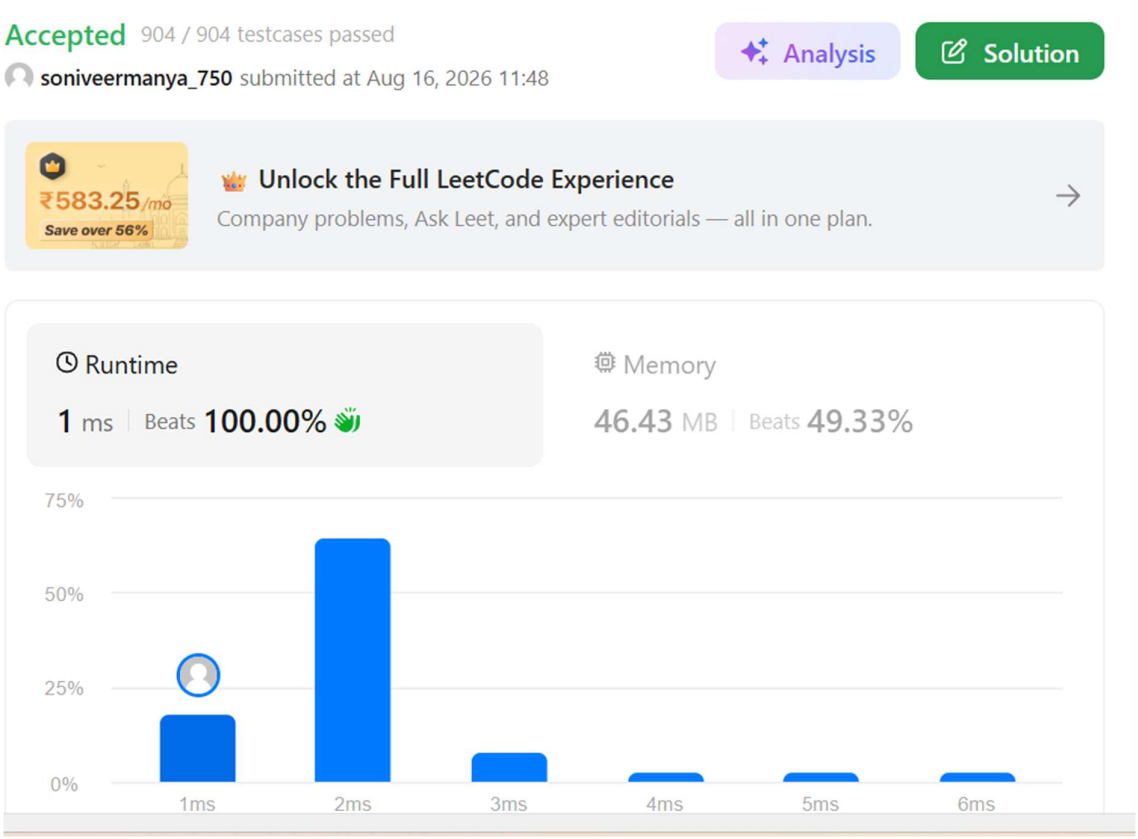
            for (int j = 0; j < baskets.length; j++) {

                if (baskets[j] >= fruits[i]) {
                    baskets[j] = -1;
                    placed = true;
                    break;
                }
            }

            if (!placed) {
                unplaced++;
            }
        }

        return unplaced;
    }
}
```

Click to collapse the range.



6.Best Time to Buy and Sell Stock

```java
class Solution {
    public int maxProfit(int[] prices) {

        int n = prices.length;

        int[] left = new int[n];
        int[] right = new int[n];

        int minPrice = prices[0];

        for (int i = 1; i < n; i++) {
            minPrice = Math.min(minPrice, prices[i]);
            left[i] = Math.max(left[i - 1], prices[i] - minPrice);
        }

        int maxPrice = prices[n - 1];

        for (int i = n - 2; i >= 0; i--) {
            maxPrice = Math.max(maxPrice, prices[i]);
            right[i] = Math.max(right[i + 1], maxPrice - prices[i]);
        }

        int answer = 0;

        for (int i = 0; i < n; i++) {
            answer = Math.max(answer, left[i] + right[i]);
        }

        return answer;
    }
}
```

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

1120 / 1120

Attempts : Correct / Total

2 / 3

Accuracy : 66%

Time Taken

0.24

7.Two Sum

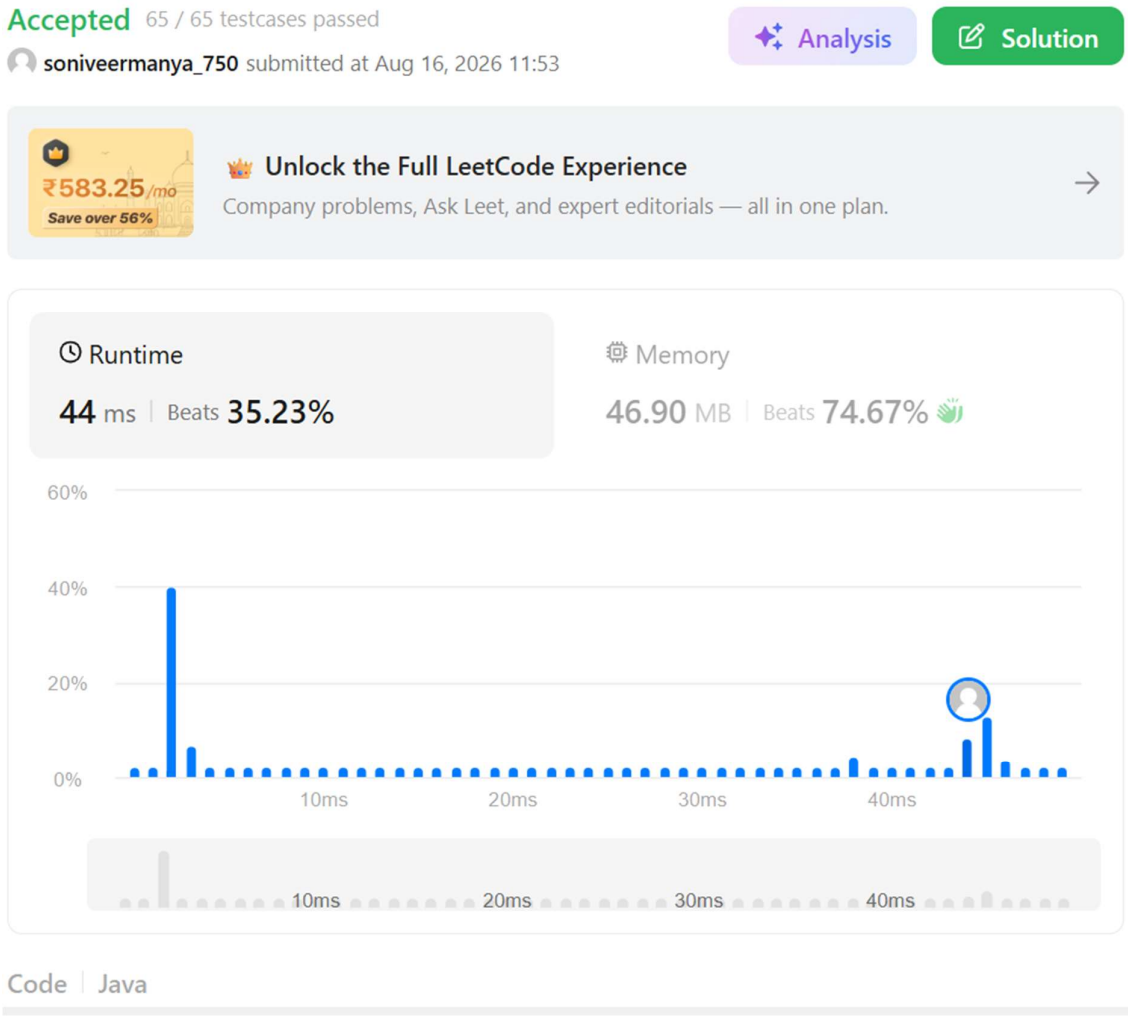
```java
class Solution {
    public int[] twoSum(int[] nums, int target) {

        for (int i = 0; i < nums.length; i++) {

            for (int j = i + 1; j < nums.length; j++) {

                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }

        return new int[]{};
    }
}
```



8. How Many Numbers Are Smaller Than the Current Number

```java
class Solution {
    public int[] smallerNumbersThanCurrent(int[] nums) {

        int[] ans = new int[nums.length];

        for (int i = 0; i < nums.length; i++) {

            int count = 0;

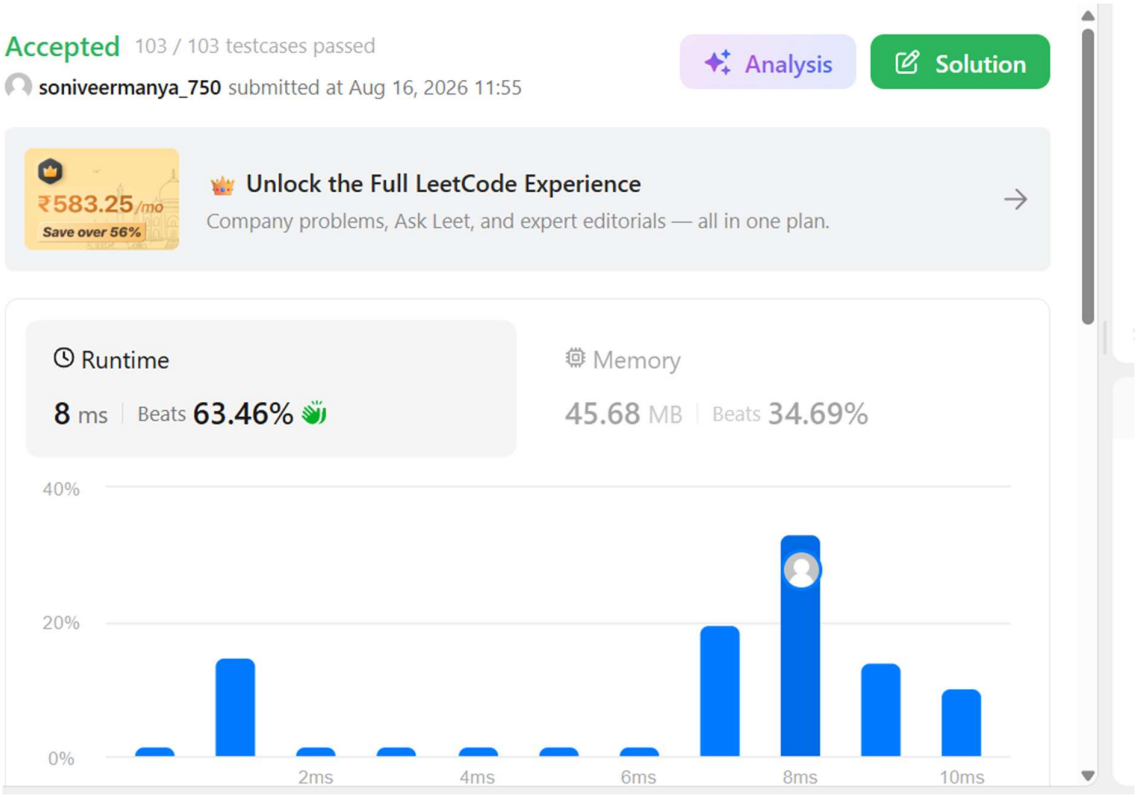
            for (int j = 0; j < nums.length; j++) {

                if (nums[j] < nums[i]) {
                    count++;
                }
            }

            ans[i] = count;
        }

        return ans;
    }
}
```

ved



9. Maximum Product of Three Numbers

```java
class Solution {
    public int maximumProduct(int[] nums) {

        int max1 = Integer.MIN_VALUE;
        int max2 = Integer.MIN_VALUE;
        int max3 = Integer.MIN_VALUE;

        int min1 = Integer.MAX_VALUE;
        int min2 = Integer.MAX_VALUE;

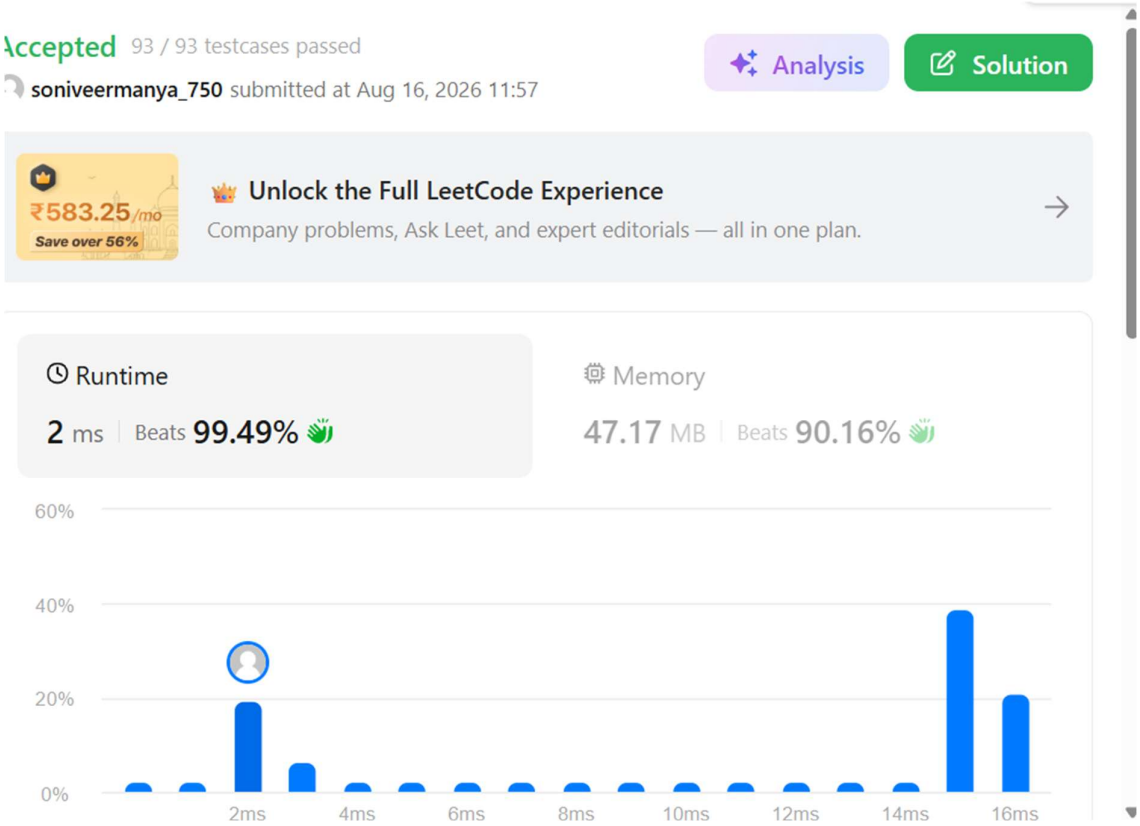
        for (int num : nums) {

            // Find 3 largest
            if (num >= max1) {
                max3 = max2;
                max2 = max1;
                max1 = num;
            }
            else if (num >= max2) {
                max3 = max2;
                max2 = num;
            }
            else if (num > max3) {
```

```
            else if (num >= max2) {
                max3 = max2;
                max2 = num;
            }
            else if (num > max3) {
                max3 = num;
            }

            // Find 2 smallest
            if (num <= min1) {
                min2 = min1;
                min1 = num;
            }
            else if (num < min2) {
                min2 = num;
            }
        }

        return Math.max(max1 * max2 * max3,
                        min1 * min2 * max1);
    }
}
```



10. Remove Duplicates from Sorted Array

```java
class Solution {
    public int removeDuplicates(int[] nums) {

        int j = 1;

        for (int i = 1; i < nums.length; i++) {

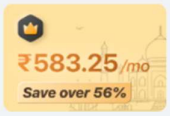
            if (nums[i] != nums[i - 1]) {
                nums[j] = nums[i];
                j++;
            }
        }

        return j;
    }
}
```

Accepted 362 / 362 testcases passed

soniveermanya_750 submitted at Aug 16, 2026 11:58

Analysis Solution



Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

0 ms | Beats 100.00%

Memory

46.80 MB | Beats 43.89%

